Jira - Jira DC SQL Tables

Jira DC. SQL tables to get all available data for filters, boards, etc

In **Jira Data Center**, filters and boards are persisted across several core tables. Below is a precise mapping of where to retrieve **all relevant metadata**, including ownership, permissions, JQL, and board configuration.

1️⃣ Filters (Saved Searches)

Core Filter Table

`searchrequest`

Primary table storing saved filters.

Key columns:

- `id`
- `filtername`
- `authorname`
- `description`
- `reqcontent` → JQL (stored as XML)
- `favourite_count`

Example:

```
SELECT id,
       filtername,
       authorname,
       reqcontent
FROM searchrequest;
```

Filter Permissions

`sharepermissions`

Controls sharing (groups, projects, global, etc.).

Important columns:

- `entityid` → `searchrequest.id`
- `entitytype` = 'SearchRequest'
- `sharetype` (group, project, global, loggedin)
- `param1` (e.g., group name, project id)

Example:

```
SELECT sr.id,
       sr.filtername,
       sp.sharetype,
       sp.param1
FROM searchrequest sr
LEFT JOIN sharepermissions sp
       ON sr.id = sp.entityid
WHERE sp.entitytype = 'SearchRequest';
```

Filter Favourites

`favouriteassociations`

- `entityid` → filter ID
- `entitytype` = 'SearchRequest'
- `username`

2️⃣ Boards (Agile Boards)

Boards in Jira DC are part of **Jira Software** (formerly GreenHopper).

Core Board Table

`AO_60DB71_RAPIDVIEW`

Key columns:

- `ID`

- `NAME`
- `OWNER_USER_NAME`
- `SAVED_FILTER_ID`
- `SPRINT_SUPPORT`

Example:

```
SELECT rv.ID,
       rv.NAME,
       rv.OWNER_USER_NAME,
       rv.SAVED_FILTER_ID,
       sr.filtername
FROM AO_60DB71_RAPIDVIEW rv
LEFT JOIN searchrequest sr
       ON rv.SAVED_FILTER_ID = sr.id;
```

Board Columns Configuration

`AO_60DB71_COLUMN`

- `RAPID_VIEW_ID`
- `NAME`
- `STATUS_ID`

Board–Sprint Association

`AO_60DB71_SPRINT`

- `ID`
- `RAPID_VIEW_ID`
- `NAME`
- `STATE`
- `START_DATE`
- `END_DATE`

Quick Filters

`AO_60DB71_QUICKFILTER`

- `RAPID_VIEW_ID`
- `NAME`
- `JQL`

3️⃣ Dashboards

If you also want dashboards:

Core Dashboard Table

`portalpage`

- `id`
- `pagename`
- `username`
- `description`

Permissions:
Same `sharepermissions` table with:

```
entitytype = 'PortalPage'
```

4️⃣ Complete "Everything Together" Query (Boards + Filters + Permissions)

```sql
SELECT rv.ID                AS board_id,
       rv.NAME              AS board_name,
       rv.OWNER_USER_NAME   AS board_owner,
       sr.id                AS filter_id,
       sr.filtername        AS filter_name,
       sr.authorname        AS filter_owner,
       sp.sharetype,
       sp.param1
FROM AO_60DB71_RAPIDVIEW rv
LEFT JOIN searchrequest sr
       ON rv.SAVED_FILTER_ID = sr.id
LEFT JOIN sharepermissions sp
       ON sr.id = sp.entityid
      AND sp.entitytype = 'SearchRequest';
```

5️⃣ Additional Related Tables (Advanced)

Depending on how deep you want to go:

Purpose	Table
Filter subscriptions	`filtersubscription`
User table	`cwd_user`
Groups	`cwd_group`
Board properties	`AO_60DB71_RAPIDVIEW_PROPERTY`
Swimlanes	`AO_60DB71_SWIMLANE`
Card colors	`AO_60DB71_CARDCOLOR`
Ranking	`AO_60DB71_RANK`

⚠️ Important Notes (Jira DC Internals)

- `reqcontent` is stored as XML — you may need to extract the `<jql>` node.
- ActiveObjects (AO_) tables are version-dependent. Prefix (e.g., `AO_60DB71_`) may differ if plugin version changes.
- Direct DB queries are read-only recommended. Atlassian does not support write operations.
- Large instances may require indexing considerations (these tables grow large).

If You Need a Full Instance Inventory

For a complete governance export, I typically pull:

1. All filters + permissions
2. All boards + linked filters
3. All dashboards + permissions
4. Orphaned filters (no boards)
5. Boards referencing deleted filters

If you'd like, I can generate a full audit SQL package tailored for a large Jira DC instance (since you likely work with Jira programmatically given your Forge background).

Generate a full audit SQL package for a large jira DC instance

Below is a **production-grade audit SQL package** for a large **Jira Data Center** instance.

It is structured for:

- Governance / SOX-style audits
- Detecting orphaned artifacts
- Identifying permission exposure risks
- Capacity / usage analysis
- Cleanup candidates

Assumptions:

- Jira DC 8.x / 9.x
- PostgreSQL (syntax compatible with minor edits for Oracle / SQL Server)
- Read-only execution
- Jira Software installed

0️⃣ Performance Preparation (Important for Large Instances)

For multi-million row instances:

```
SET statement_timeout = '10min';
SET work_mem = '256MB';
```

Indexes that must exist (normally present):

- searchrequest(id)
- AO_60DB71_RAPIDVIEW(ID)
- sharepermissions(entityid, entitytype)
- favouriteassociations(entityid, entitytype)

1️⃣ FULL FILTER INVENTORY

1.1 Filters with Owners, JQL, Favourites

```
SELECT
    sr.id                                   AS filter_id,
    sr.filtername                           AS filter_name,
    sr.authorname                           AS filter_owner,
    cu.display_name                         AS owner_display_name,
    sr.favourite_count,
    sr.reqcontent                           AS raw_jql_xml
FROM searchrequest sr
LEFT JOIN cwd_user cu
       ON lower(cu.user_name) = lower(sr.authorname)
ORDER BY sr.id;
```

1.2 Filter Permissions (Exposure Risk)

```
SELECT
    sr.id                AS filter_id,
    sr.filtername,
    sp.sharetype,
    sp.param1            AS permission_param
FROM searchrequest sr
LEFT JOIN sharepermissions sp
       ON sr.id = sp.entityid
WHERE sp.entitytype = 'SearchRequest'
ORDER BY sr.id;
```

High-risk cases:

- `sharetype = 'global'`
- `sharetype = 'loggedin'`

1.3 Orphaned Filters (Not Used by Boards)

```
SELECT sr.id,
       sr.filtername,
       sr.authorname
FROM searchrequest sr
LEFT JOIN AO_60DB71_RAPIDVIEW rv
       ON rv.SAVED_FILTER_ID = sr.id
WHERE rv.ID IS NULL;
```

1.4 Filters Referencing Deleted Owners

```sql
SELECT sr.id,
       sr.filtername,
       sr.authorname
FROM searchrequest sr
LEFT JOIN cwd_user cu
       ON lower(cu.user_name) = lower(sr.authorname)
WHERE cu.user_name IS NULL;
```

2️⃣ FULL BOARD AUDIT

2.1 Board Core Inventory

```sql
SELECT
    rv.ID                       AS board_id,
    rv.NAME                     AS board_name,
    rv.OWNER_USER_NAME,
    sr.id                       AS filter_id,
    sr.filtername,
    sr.authorname               AS filter_owner
FROM AO_60DB71_RAPIDVIEW rv
LEFT JOIN searchrequest sr
       ON rv.SAVED_FILTER_ID = sr.id
ORDER BY rv.ID;
```

2.2 Boards Referencing Missing Filters

```sql
SELECT rv.ID,
       rv.NAME,
       rv.SAVED_FILTER_ID
FROM AO_60DB71_RAPIDVIEW rv
LEFT JOIN searchrequest sr
       ON rv.SAVED_FILTER_ID = sr.id
WHERE sr.id IS NULL;
```

2.3 Board Columns + Status Mapping

```sql
SELECT
    rv.ID                   AS board_id,
    rv.NAME                 AS board_name,
    col.NAME                AS column_name,
    col.STATUS_ID
FROM AO_60DB71_COLUMN col
JOIN AO_60DB71_RAPIDVIEW rv
  ON col.RAPID_VIEW_ID = rv.ID
ORDER BY rv.ID;
```

2.4 Quick Filters Inventory

```sql
SELECT
    rv.ID                AS board_id,
    rv.NAME              AS board_name,
    qf.NAME              AS quick_filter_name,
    qf.JQL
FROM AO_60DB71_QUICKFILTER qf
JOIN AO_60DB71_RAPIDVIEW rv
  ON qf.RAPID_VIEW_ID = rv.ID
ORDER BY rv.ID;
```

2.5 Sprints Inventory

```sql
SELECT
    sp.ID,
    sp.RAPID_VIEW_ID,
    rv.NAME AS board_name,
    sp.NAME,
    sp.STATE,
    sp.START_DATE,
    sp.END_DATE
FROM AO_60DB71_SPRINT sp
LEFT JOIN AO_60DB71_RAPIDVIEW rv
       ON sp.RAPID_VIEW_ID = rv.ID
ORDER BY sp.ID;
```

3️⃣ DASHBOARD AUDIT

3.1 Dashboard Inventory

```sql
SELECT
    pp.id,
    pp.pagename,
    pp.username,
    cu.display_name,
    pp.favourite_count
FROM portalpage pp
LEFT JOIN cwd_user cu
       ON lower(cu.user_name) = lower(pp.username)
ORDER BY pp.id;
```

3.2 Dashboard Permissions

```sql
SELECT
    pp.id,
    pp.pagename,
    sp.sharetype,
    sp.paraml
FROM portalpage pp
LEFT JOIN sharepermissions sp
       ON pp.id = sp.entityid
WHERE sp.entitytype = 'PortalPage'
ORDER BY pp.id;
```

4️⃣ USER ACTIVITY & OWNERSHIP RISK

4.1 Boards Owned by Inactive Users

```sql
SELECT
    rv.ID,
    rv.NAME,
    rv.OWNER_USER_NAME
FROM AO_60DB71_RAPIDVIEW rv
LEFT JOIN cwd_user cu
       ON lower(cu.user_name) = lower(rv.OWNER_USER_NAME)
WHERE cu.active = 0;
```

4.2 Filters Owned by Inactive Users

```sql
SELECT
    sr.id,
    sr.filtername,
    sr.authorname
FROM searchrequest sr
JOIN cwd_user cu
  ON lower(cu.user_name) = lower(sr.authorname)
WHERE cu.active = 0;
```

5️⃣ PERMISSION EXPOSURE REPORT (High Risk Artifacts)

Public Filters + Boards Using Them

```sql
SELECT
    rv.ID AS board_id,
    rv.NAME AS board_name,
    sr.id AS filter_id,
    sr.filtername,
    sp.sharetype
FROM AO_60DB71_RAPIDVIEW rv
JOIN searchrequest sr
  ON rv.SAVED_FILTER_ID = sr.id
JOIN sharepermissions sp
  ON sr.id = sp.entityid
WHERE sp.entitytype = 'SearchRequest'
  AND sp.sharetype IN ('global','loggedin');
```

6️⃣ STORAGE SIZE ANALYSIS (PostgreSQL Only)

```sql
SELECT
    relname AS table_name,
    pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

7️⃣ SUMMARY COUNTS (Executive View)

```sql
SELECT
  (SELECT COUNT(*) FROM searchrequest)           AS total_filters,
  (SELECT COUNT(*) FROM AO_60DB71_RAPIDVIEW)     AS total_boards,
  (SELECT COUNT(*) FROM portalpage)              AS total_dashboards,
  (SELECT COUNT(*) FROM AO_60DB71_SPRINT)        AS total_sprints;
```

8️⃣ Optional: Materialized Audit View (For Recurring Reporting)

```sql
CREATE MATERIALIZED VIEW jira_audit_boards AS
SELECT
    rv.ID AS board_id,
    rv.NAME AS board_name,
    rv.OWNER_USER_NAME,
    sr.filtername,
    sr.authorname,
    sp.sharetype
FROM AO_60DB71_RAPIDVIEW rv
LEFT JOIN searchrequest sr
       ON rv.SAVED_FILTER_ID = sr.id
LEFT JOIN sharepermissions sp
       ON sr.id = sp.entityid
      AND sp.entitytype = 'SearchRequest';
```

Refresh with:

```sql
REFRESH MATERIALIZED VIEW jira_audit_boards;
```

9️⃣ Recommended Audit Workflow for Large Enterprises

For large Jira DC (>5k users):

1. Export all results to CSV
2. Classify:
    - Public exposure
    - Orphaned artifacts
    - Inactive ownership
3. Cross-match with project category governance
4. Feed into cleanup automation

Given your background building apps for Jira/Confluence, this dataset is ideal for building:

- Governance dashboards
- Ownership transfer automation
- Stale artifact cleanup jobs
- Public exposure alerting

If you want, I can next generate:

- A consolidated single audit script
- A PowerBI / Tableau schema mapping
- An automated cleanup SQL recommendation set
- A Forge/REST-based alternative (fully supported approach)

Tell me your target outcome (security audit, cleanup, migration, compliance, cost reduction).

Jira DC JQL - give me script to verify JQL quality, efficiency

In **Jira Data Center**, there is no built-in "JQL quality analyzer," but you can audit JQL quality and performance risk systematically by analyzing:

- Saved filters (`searchrequest`)
- Board quick filters
- Subscriptions
- Problematic JQL patterns
- Missing index usage signals
- High-cost operators

Below is a **database-driven JQL quality audit package** designed for large DC instances.

This is read-only and safe.

🎯 What This Script Detects

Flags JQL that:

- ❌ Missing `project =` (cross-instance scans)
- ❌ Missing `issuetype =`
- ❌ Uses `text ~`
- ❌ Uses `ORDER BY` on non-indexed fields
- ❌ Uses `!=` or `NOT IN`
- ❌ Uses `OR` excessively
- ❌ Uses leading wildcards (`~ "%abc"`)
- ❌ No status constraint
- ❌ Large IN lists
- ❌ Uses `WAS` (history scan)
- ❌ Uses `CHANGED`
- ❌ Uses date functions without bounds

1️⃣ Extract Clean JQL (PostgreSQL)

JQL is stored as XML inside `searchrequest.reqcontent`.

```
WITH extracted_jql AS (
    SELECT
        sr.id,
        sr.filtername,
        sr.authorname,
        regexp_replace(
            sr.reqcontent,
            '.*<jql>(.*)</jql>.*',
            '\1'
        ) AS jql
    FROM searchrequest sr
)
SELECT * FROM extracted_jql;
```

2️⃣ JQL Risk Classification Engine

This assigns a risk score.

```
WITH extracted_jql AS (
    SELECT
        sr.id,
        sr.filtername,
        sr.authorname,
        lower(
            regexp_replace(
                sr.reqcontent,
                '.*<jql>(.*)</jql>.*',
                '\1'
            )
        ) AS jql
    FROM searchrequest sr
),
scored AS (
SELECT
    id,
    filtername,
    authorname,
    jql,

    /* Risk Indicators */
    CASE WHEN jql NOT LIKE '%project %' THEN 1 ELSE 0 END AS no_project,
    CASE WHEN jql LIKE '%text ~%' THEN 1 ELSE 0 END AS text_search,
    CASE WHEN jql LIKE '% != %' THEN 1 ELSE 0 END AS not_equal,
    CASE WHEN jql LIKE '% not in %' THEN 1 ELSE 0 END AS not_in,
    CASE WHEN jql LIKE '% was %' THEN 1 ELSE 0 END AS history_was,
    CASE WHEN jql LIKE '% changed %' THEN 1 ELSE 0 END AS history_changed,
    CASE WHEN jql LIKE '% or %' THEN 1 ELSE 0 END AS uses_or,
    CASE WHEN jql NOT LIKE '%status %' THEN 1 ELSE 0 END AS no_status,
```

3️⃣ Identify High-Risk Filters Used by Boards

```
WITH risky AS (
    SELECT
    FROM
    -- reuse previous scoring CTE here
    WHERE
)
SELECT
    rv.ID AS board_id,
    rv.NAME AS board_name,
    sr.filtername,
    r.total_risk_score
FROM AO_60DB71_RAPIDVIEW rv
JOIN searchrequest sr
  ON rv.SAVED_FILTER_ID = sr.id
JOIN risky r
  ON r.id = sr.id
ORDER BY r.total_risk_score DESC;
```

These are **performance-impacting boards**.

4️⃣ Detect Filters Missing Project Scoping (Worst Offenders)

```
SELECT
    sr.id,
    sr.filtername,
    sr.authorname
FROM searchrequest sr
WHERE lower(sr.reqcontent) NOT LIKE '%project %';
```

Unscoped filters in large DC instances cause full index scans.

5️⃣ Detect Expensive History Queries

```
SELECT
    sr.id,
    sr.filtername
FROM searchrequest sr
WHERE lower(sr.reqcontent) LIKE '% was %'
   OR lower(sr.reqcontent) LIKE '% changed %';
```

These hit change history tables (`changegroup`, `changeitem`) heavily.

6️⃣ Quick Filters (Often Forgotten but Expensive)

```
SELECT
    rv.NAME AS board_name,
    qf.NAME AS quick_filter_name,
    qf.JQL
FROM AO_60DB71_QUICKFILTER qf
JOIN AO_60DB71_RAPIDVIEW rv
  ON rv.ID = qf.RAPID_VIEW_ID
WHERE lower(qf.JQL) LIKE '% text ~%'
   OR lower(qf.JQL) LIKE '% was %'
   OR lower(qf.JQL) NOT LIKE '%project %';
```

Quick filters execute repeatedly in UI — high impact.

7️⃣ Subscription Risk (Nightly Load Amplification)

```
SELECT
    fs.id,
    sr.filtername,
    sr.authorname,
    fs.cron_expression
FROM filtersubscription fs
JOIN searchrequest sr
  ON fs.filter_id = sr.id;
```

Combine with risk score to find:

- Heavy filters
- Running on cron
- Emailing thousands of users

8️⃣ Executive Summary

```
SELECT
  COUNT(*) FILTER (WHERE lower(reqcontent) LIKE '%text ~%') AS text_queries,
  COUNT(*) FILTER (WHERE lower(reqcontent) LIKE '% was %') AS history_queries,
  COUNT(*) FILTER (WHERE lower(reqcontent) NOT LIKE '%project %') AS unscoped_queries,
  COUNT(*) AS total_filters
FROM searchrequest;
```

🧮 Interpreting Risk Scores

Score	Meaning
0–1	Safe
2–3	Needs review

Score	Meaning
4–5	Performance risk
6+	Likely cluster-impacting

In instances > 10M issues, anything without `project =` is a red flag.

🧠 Efficiency Best Practices (Jira DC Internals)

High-performance JQL:

- Always include `project =`
- Always include `issuetype =`
- Prefer `=` over `!=`
- Avoid leading wildcards
- Avoid `WAS` unless scoped tightly
- Avoid large `IN` lists (>100 values)
- Use indexed fields (project, issuetype, status, resolution, created, updated, assignee)

⚠️ Important

Database analysis only tells you structural risk.

True performance requires:

- Query profiling via `atlassian-jira-slow-queries.log`
- Checking `jira.search.views.default.max`
- Monitoring `com.atlassian.jira.issue.search.providers`

If You Want Advanced Mode

I can generate:

- A PL/pgSQL function that automatically scores and stores results
- A live monitoring query against slow query logs
- A REST-based JQL profiler script
- A full governance scoring model with weighting
- A Forge app concept for continuous JQL linting

Tell me your Jira size (issue count + users) and your objective (cluster performance vs governance vs cleanup).

Generate rest-based JQL profile script

Below is a **REST-based JQL profiling script** for **Jira Data Center** that:

- Extracts all saved filters
- Executes each filter via REST
- Measures response time
- Captures result count
- Scores structural risk
- Flags performance risks
- Produces a ranked output

This is safe and supported (uses official REST APIs).

🔎 What This Script Profiles

For each saved filter:

- Execution time (ms)
- Result count
- Presence of:
  - Unscoped queries (no `project =`)
  - `text ~`
  - `WAS` / `CHANGED`
  - `!=` / `NOT IN`
  - Excessive `OR`
  - Missing `status`
- Overall risk score

⚙️ Requirements

- Python 3.9+
- `requests` library
- Admin or global filter read access
- API user with permission to search

Install dependency:

```
pip install requests
```

🧠 Script: JQL Profiler (Jira Data Center)

```
import requests
import time
import re
from requests.auth import HTTPBasicAuth

# ========================
# CONFIGURATION
# ========================
JIRA_BASE_URL = "https://your-jira-dc-instance"
USERNAME = "admin"
PASSWORD = "password"
VERIFY_SSL = True

HEADERS = {
    "Accept": "application/json",
    "Content-Type": "application/json"
}

auth = HTTPBasicAuth(USERNAME, PASSWORD)

# ========================
# JQL RISK SCORING
# ========================
def score_jql(jql: str):
    jql_lower = jql.lower()

    score = 0
    flags = []

    if "project " not in jql_lower:
        score += 2
        flags.append("NO_PROJECT_SCOPE")
    if "text ~" in jql_lower:
        score += 2
        flags.append("TEXT_SEARCH")

    if " was " in jql_lower:
        score += 3
        flags.append("HISTORY_WAS")
```

📊 What This Produces

For each filter:

- Execution time (real cluster cost)

- Result count
- Risk score
- Flags explaining why

You can redirect output to CSV if needed.

🔥 For Large Jira DC (>5M Issues)

Modify `profile_jql()` to test deeper pagination cost:

This forces actual issue retrieval (heavier but realistic).

🛡️ Production Safety Tips

- Run off-peak
- Throttle (add `time.sleep(0.2)`)
- Use a dedicated profiling account
- Avoid running on peak index rebuild windows

🚀 Next-Level Options

If you're operating a serious enterprise instance, I can generate:

- Parallelized profiler (ThreadPoolExecutor)
- HTML dashboard report
- CSV export version
- Risk-weighted governance scoring
- Slack alert integration
- Automatic remediation suggestions
- A Forge-based JQL linting app architecture

Given your engineering background, a continuous JQL linting service with nightly scoring would be very realistic.

Tell me:

- Issue count?
- Node count?
- DB type?
- Performance pain or governance focus?

```
    print(r)
if __name__ == "__main__":
    main()
```

Headlines in order of most impactful items first

🚨 Most Impactful JQL Risk Patterns (Highest Cluster Impact First)

1️⃣ Unscoped Queries (Missing `project =`)

Impact: Full index scan across entire issue corpus
Why critical: On large DC instances (millions of issues), this is the single biggest performance amplifier.
Risk indicator: No `project =` clause.

2️⃣ History Operators (`WAS`, `CHANGED`)

Impact: Heavy joins on `changegroup` and `changeitem` tables
Why critical: Bypasses optimized issue index; scales poorly with issue age and volume.
Risk indicator: `WAS`, `CHANGED`, `DURING`, `BEFORE`, `AFTER`

3️⃣ `text ~` (Lucene Full-Text Search)

Impact: Expensive text index evaluation
Why critical: Frequently used in dashboards and boards; multiplies UI refresh cost.
Risk indicator: `text ~`, summary ~, description ~

4️⃣ Leading Wildcards (`~ "%abc"`)

Impact: Forces reverse scan of text index
Why critical: Negates index prefix optimization.
Risk indicator: `%` at beginning of search term.

5️⃣ Board Filters Using Risky JQL

Impact: Continuous execution on every board load, sprint view, backlog view
Why critical: Repeated execution multiplies cost cluster-wide.
Scope: Filters linked via `AO_60DB71_RAPIDVIEW`

6️⃣ Large `IN (...)` Lists

Impact: Query planner expansion overhead
Why critical: Very common in automation-created filters.
Threshold: >100 values in `IN`

7️⃣ Excessive `OR` Conditions

Impact: Query branch explosion in Lucene
Why critical: Degrades query rewrite performance
Risk indicator: 3+ `OR` operators

8️⃣ Negative Conditions (`!=`, `NOT IN`)

Impact: Reverse matching (anti-join semantics)
Why critical: Forces broader evaluation before exclusion
Safer alternative: Positive inclusion where possible

9️⃣ Missing `status` Constraint

Impact: Expands working result set
Why important: Status is indexed and narrows result sets efficiently

---

🔟 Scheduled Subscriptions Running Heavy JQL

Impact: Nightly cluster spikes
Why critical: Amplifies impact during off-hours; often unnoticed

---

Executive Impact Order (Enterprise View)

Rank	Risk Type	Cluster Impact
1	No project scope	🔴 Severe
2	History operators	🔴 Severe

Rank	Risk Type	Cluster Impact
3	Board-linked risky filters	🔴 Severe
4	`text ~` usage	🟠 High
5	Leading wildcards	🟠 High
6	Large IN lists	🟡 Moderate
7	Excessive OR	🟡 Moderate
8	Negative operators	🟡 Moderate
9	No status constraint	🟢 Low–Moderate
10	Scheduled heavy filters	🔴 Severe (time-based)

If you want, I can now provide:

- A weighted scoring model tuned for >10M issue instances
- A board-specific performance impact ranking
- A remediation rewrite guide (before/after JQL examples)
- A production-grade parallel profiler optimized for multi-node Jira DC

Tell me your instance size and whether you're optimizing for cluster stability or governance hygiene.

A production-grade parallel profiler optimized for multi-node Jira DC

Below is a **production-grade, parallel JQL profiler** engineered for **multi-node Jira Data Center** clusters.

Design goals:

- Controlled parallelism (prevents node overload)
- Node-aware load distribution
- Retry + backoff logic
- Rate limiting
- Timeout enforcement
- CSV output for governance
- Deterministic risk scoring
- Safe for large instances (10M+ issues)

🧠 Architecture Strategy for Jira DC

In a clustered DC environment:

- `/rest/api/2/search` executes on the node receiving the request
- Sticky sessions may bias traffic to one node
- Boards and dashboards amplify risky filters
- Slow queries affect search threads, not DB only

This profiler:

- Uses a thread pool
- Limits concurrency
- Supports round-robin node targeting
- Measures:
  - Latency
  - Result size
  - HTTP errors
  - Risk score

⚙️ Requirements

```
pip install requests tqdm
```

Python 3.10+

🔧 CONFIGURATION SECTION

```
# ========================
# CONFIGURATION
# ========================

JIRA_NODES = [
    "https://jira-node1.company.com",
    "https://jira-node2.company.com",
    "https://jira-node3.company.com"
]

USERNAME = "profiler_user"
PASSWORD = "password"

MAX_WORKERS = 6              # Safe default for large DC
REQUEST_TIMEOUT = 30         # seconds
RATE_LIMIT_DELAY = 0.1       # seconds between submissions
VERIFY_SSL = True
MAX_RESULTS_PROFILE = 0      # 0 = count only, 100 = deeper cost simulation
RETRIES = 2
```

🚀 Production-Grade Parallel Profiler

```python
import requests
import time
import re
import csv
import itertools
from concurrent.futures import ThreadPoolExecutor, as_completed
from requests.auth import HTTPBasicAuth
from tqdm import tqdm

# ========================
# SESSION SETUP
# ========================

auth = HTTPBasicAuth(USERNAME, PASSWORD)
node_cycle = itertools.cycle(JIRA_NODES)

session = requests.Session()
session.auth = auth
session.headers.update({
    "Accept": "application/json",
    "Content-Type": "application/json"
})

# ========================
# RISK SCORING
# ========================

def score_jql(jql):
    jql_lower = jql.lower()
    score = 0
    flags = []

    if " project " in jql_lower:
        score += 3
        flags.append("NO_PROJECT")

    if " was " in jql_lower:
        score += 3
        flags.append("WAS")

    if " changed " in jql_lower:
        score += 3
        flags.append("CHANGED")

    if "text ~" in jql_lower:
        score += 2
        flags.append("TEXT")

    if re.search(r"~\s*['\"]*%[^'\"]", jql_lower):
        score += 2
        flags.append("LEADING_WILDCARD")

    if " or " in jql_lower:
        score += 1
        flags.append("OR_HEAVY")

    if " not in " in jql_lower:
        score += 1
        flags.append("NOT_IN")

    if " != " in jql_lower:
        score += 1
        flags.append("NOT_EQUAL")

    if " status " not in jql_lower:
        score += 1
        flags.append("NO_STATUS")

    return score, "|".join(flags)

# ========================
# GET FILTERS
# ========================

def fetch_filters():
    base_url = next(node_cycle)

    start = 0
    max_results = 50

    while True:
        url = f"{base_url}/rest/api/2/filter/search"
        params = {
            "startAt": start,
            "maxResults": max_results,
            "expand": "owner"
        }
        r = session.get(url, params=params, timeout=REQUEST_TIMEOUT, verify=VERIFY_SSL)
        r.raise_for_status()

        data = r.json()
        filters.extend(data.get("values", []))

        if start + max_results >= data.get("total", 0):
            break

        start += max_results

    return filters

# ========================
# PROFILE FILTER
# ========================

def profile_filter(filter_obj):
    jql = filter_obj.get("jql")
    name = filter_obj.get("name")
    fid = filter_obj.get("id")
    owner = filter_obj.get("owner", {}).get("displayName")

    if not jql:
        return None

    node = next(node_cycle)
    url = f"{node}/rest/api/2/search"
    payload = {
        "jql": jql,
```

🧮 Safe Concurrency Recommendations (Enterprise)

Instance Size	MAX_WORKERS
< 1M issues	4
1–5M issues	6
5–15M issues	8
15M+ issues	4–6 (safer)

Never exceed available search thread pool.

📊 What This Produces

For every filter:

- Node used
- Execution latency
- Result count
- Risk score
- Structural flags
- HTTP/timeout detection

🛡️ Production Safety Checklist

Before running:

- Off-peak hours
- Confirm no reindex running
- Confirm cluster healthy
- Dedicated profiling user
- Disable email notifications for user

🔬 Advanced Extensions (Enterprise-Grade)

If needed, I can extend this with:

- Board-only profiling (high impact first)
- Subscription profiling
- Percentile latency analysis
- P95/P99 breakdown
- Node imbalance detection
- Automatic rewrite suggestions
- Integration with Prometheus
- Scheduled nightly governance job
- REST-based slow query correlation
- Cluster heatmap visualization

Given your experience building Jira ecosystem tooling, the next logical step is turning this into:

- A continuous governance service
- Or a Forge-backed linting and profiling microservice

If you share:

- Issue count
- Node count
- DB type (Postgres/Oracle)
- Primary concern (performance vs governance)

I can tailor the profiler to your production topology.

```
        "maxResults": MAX_RESULTS_PROFILE,
        "fields": "key",
    }

    for attempt in range(1, RETRIES + 1):
        try:
            start_time = time.time()

            r = session.post(
                url,
                json=payload,
                timeout=REQUEST_TIMEOUT,
                verify=VERIFY_SSL
            )

            latency = round((time.time() - start_time) * 1000, 2)

            if r.status_code != 200:
                return {
                    "id": fid,
                    "name": name,
                    "owner": owner,
                    "latency_ms": latency,
                    "result_count": None,
                    "risk_score": None,
                    "flags": "HTTP_ERROR",
                    "node": node
                }

            total = r.json().get("total", 0)
            score, flags = score_jql(jql)

            return {
                "id": fid,
                "name": name,
                "owner": owner,
                "latency_ms": latency,
                "result_count": total,
                "risk_score": score,
                "flags": flags,
                "node": node
            }

        except Exception:
            if attempt == RETRIES:
                return {
                    "id": fid,
                    "name": name,
                    "owner": owner,
                    "latency_ms": None,
                    "result_count": None,
                    "risk_score": None,
                    "flags": "TIMEOUT",
                    "node": node
                }

# ========================
# MAIN
# ========================

def main():
    print("Fetching filters...")
    filters = fetch_filters()
    print(f"Total filters: {len(filters)}")

    results = []

    with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
        futures = []

        for f in filters:
            futures.append(executor.submit(profile_filter, f))
            time.sleep(RATE_LIMIT_DELAY)

        for future in tqdm(as_completed(futures), total=len(futures)):
            result = future.result()
            if result:
                results.append(result)

    # Sort by latency + risk
    results.sort(key=lambda x: (
        x["risk_score"] if x["risk_score"] else 0,
        x["latency_ms"] if x["latency_ms"] else 0
    ), reverse=True)

    # Export CSV
    with open("jira_jql_profile.csv", "w", newline="") as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=results[0].keys())
        writer.writeheader()
        writer.writerows(results)

    print("Profiling complete. Output: jira_jql_profile.csv")

if __name__ == "__main__":
    main()
```